BLACKJACK

Github Link: https://github.com/randymoore44/Assignment2

A. **Environment description.** Describe the environment's goal, state/observation space, action space, transition dynamics, reward structure, and termination conditions. Explain any state discretization or environment settings you use.

The environment's goal is for the player to get closer to 21 than the dealer without going over, either by hitting (drawing another card) or sticking (not drawing any more cards). The state/observation space consists of the player's current sum, the value of the dealer's one showing card, and a 0 or 1 indicating whether the player has an ace, which can be a 1 or 11. The action space is 0 for stick and 1 for hit. When the player chooses to hit, they receive another card, and their sum increases; this continues until they either stop or go over 21, which means they lose. When the player chooses to stick, the dealer draws cards as needed until reaching 17 or more. If the player goes over 21 or has a sum less than the dealer's, they lose. Otherwise, the player will win. A win has a reward of +1, a draw is 0, a loss is -1, and a blackjack (meaning 21 in the initial state) is +1.5. The blackjack environment has a discrete state/observation space. I use these initial environment settings to perform Q-learning to train the agent to maximize the rewards when playing blackjack:

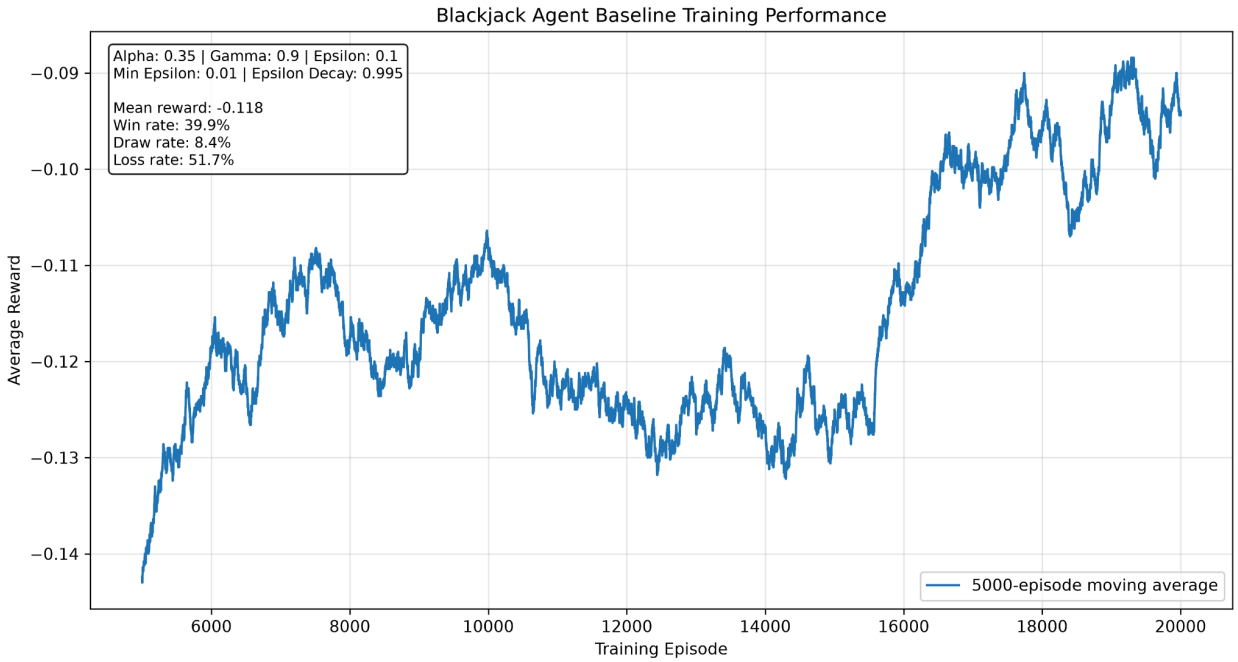
- Learning rate (0.35): Controls step size
- Discount factor (0.9): Weighs future rewards
- Exploration rate (0.1): How often to perform random actions
- Exploration decay (0.995): Rate at which we explore less
- Minimum exploration rate (0.01): Lowest exploration rate after decaying
- Episodes (20,000): The number of games carried out for training

B. **Implement Q-learning.** Write your own tabular Q-learning agent, including Q-value initialization, an ε-greedy action-selection policy, a training loop, terminal-state handling, and the update.

I implemented this in a Jupyter notebook, along with random tie-breaking, to help prevent bias.

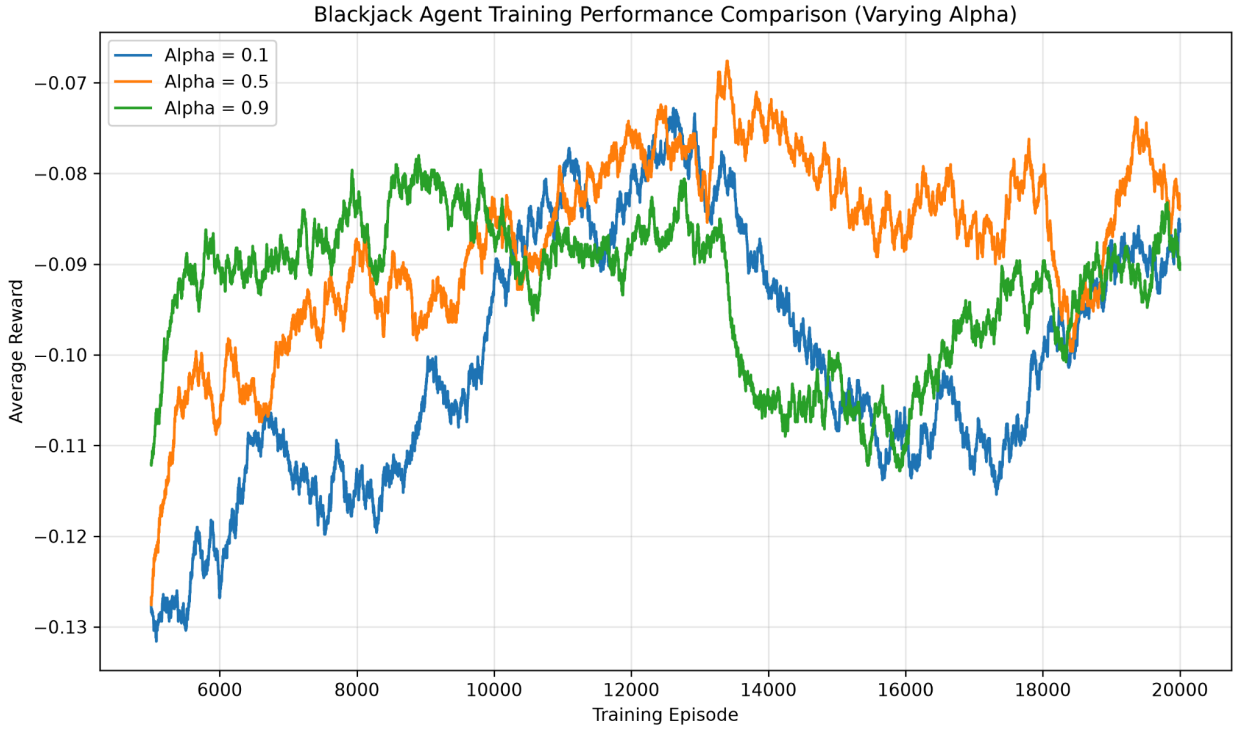
C. **Baseline training.** Choose and report a baseline configuration (α, γ, ε, number of episodes, and any exploration schedule). Train the agent and include a plot of reward versus training episode. Explain your initial results.

α = 0.35
γ = 0.9
ε = 0.1 | Decay = 0.995 | Minimum = 0.01
of Episodes = 20,000



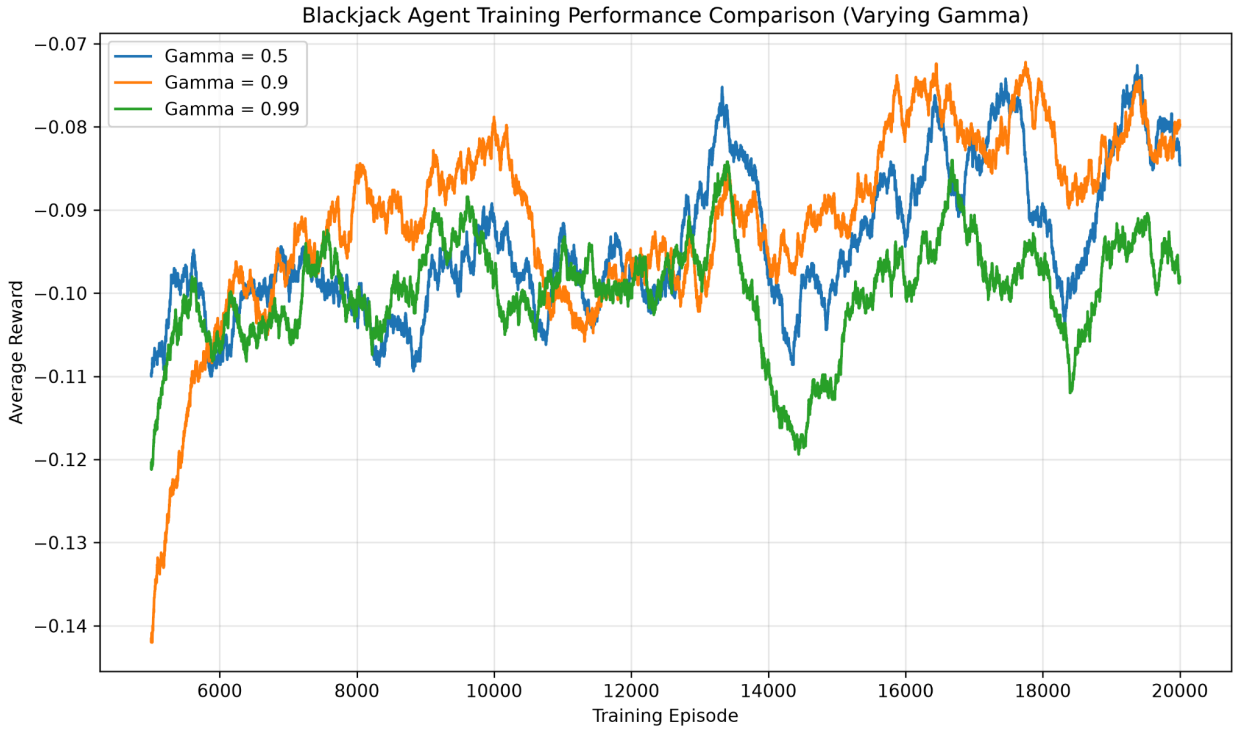
The baseline results show that the agent improves over time, but there is substantial noise. The mean reward over 20k episodes was -0.118, with a win rate of 39.9%.

D. **Learning-rate experiment.** Test at least three values of α (e.g., 0.1, 0.5, 0.9) while keeping the other settings fixed. Plot and compare learning speed, stability, and final performance. Explain your observations.



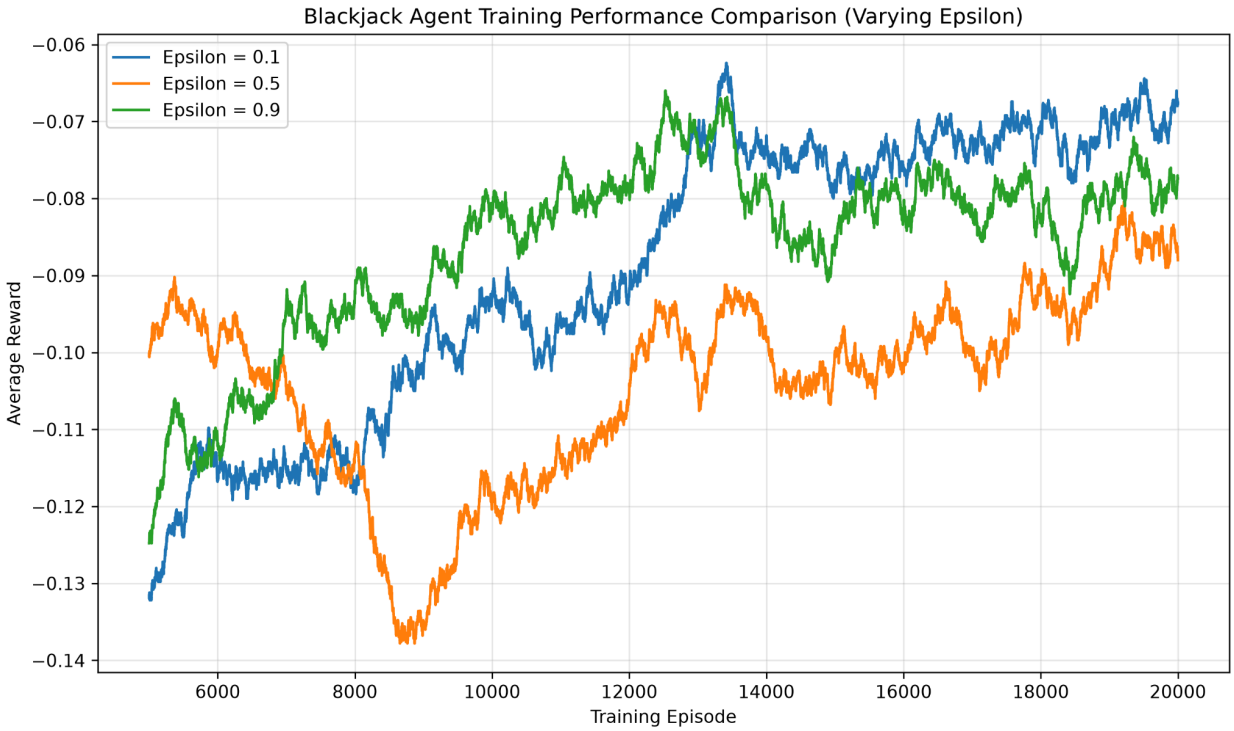
Looking at the results for varying values of alpha, I observed that alpha = 0.9 initially learns the fastest, but eventually its performance drops and it does not achieve the highest average reward. An alpha of 0.1 performs similarly, but it has the slowest initial learning rate. Alpha = 0.5 appears to reach the highest average reward and also ends with the highest of the 3. It also looks to have fewer large fluctuations.

E. **Discount-factor experiment.** Test at least three values of γ (e.g., 0.50, 0.90, 0.99), keeping the other settings fixed. Present a comparison plot and discuss the effect of valuing immediate versus future rewards.



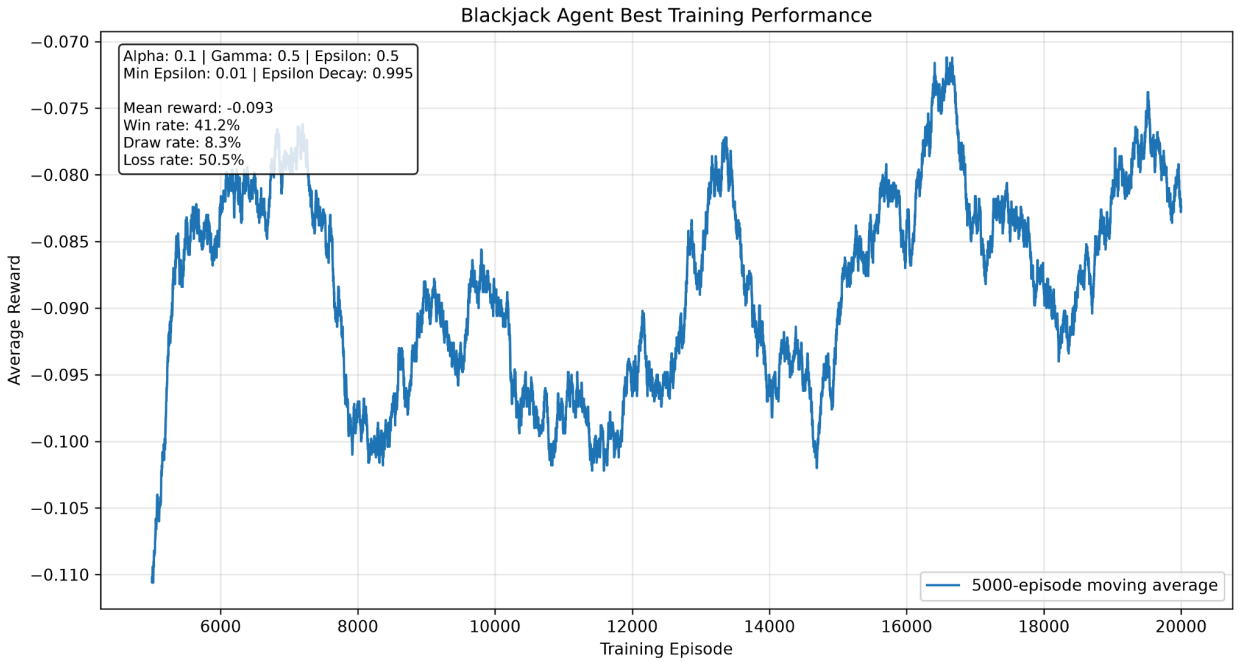
The discount factor of 0.5 seems to be the most stable of the 3, as its fluctuations are smaller than those of the others. It ends with a similar average reward to that of gamma = 0.9, but starts with a higher average reward. The higher discount factor appears to have negatively affected learning, with gamma = 0.99 yielding the worst final average reward.

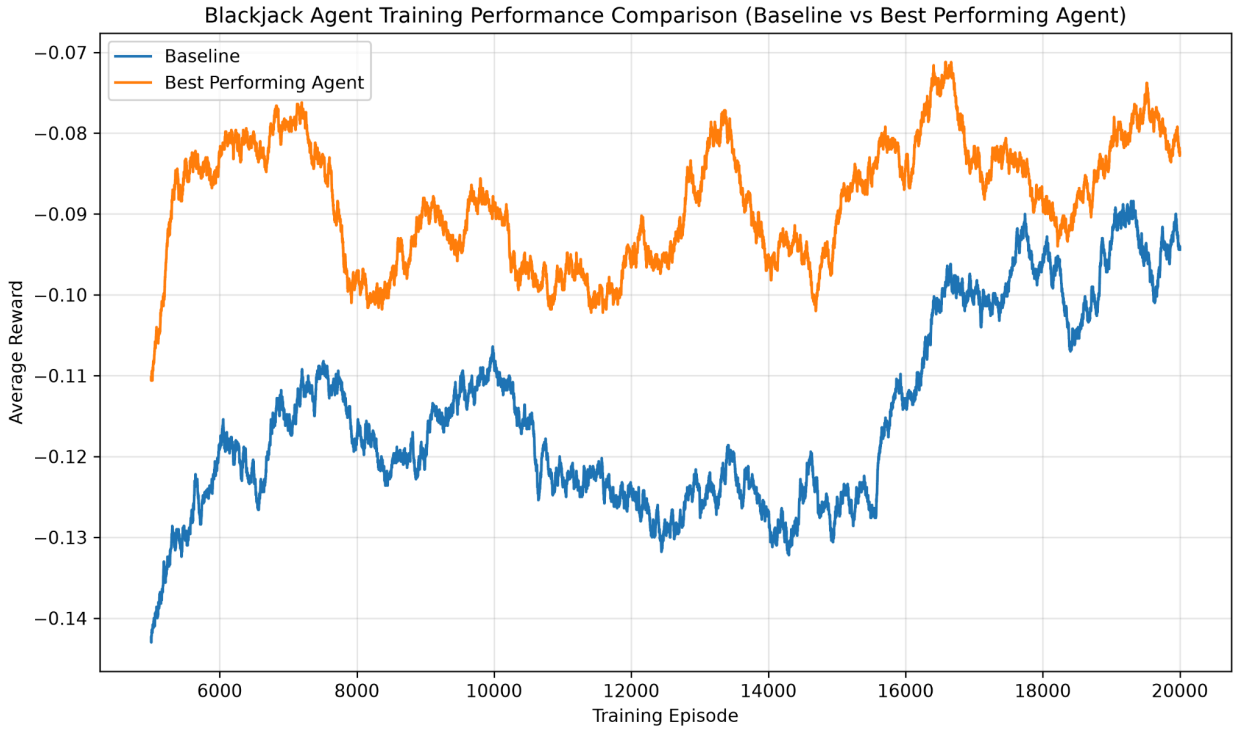
F. **One additional hyperparameter.** Vary at least three settings of one other parameter (e.g., ε, ε-decay, training episodes, or state discretization). Include a comparison plot and explain the effect on learning.



For the third varying parameter, I chose epsilon. I noticed that a lower epsilon appears to have the best impact on learning and leads to the highest ending average reward. An epsilon of 0.5 appears to cause less stability, as the average reward decreases a lot in the beginning, then slowly improves with bumps. An epsilon of 0.9 works well too and yields the second-best average reward among the varying epsilons. There are still more fluctuations as the agent explores more, but due to the randomness of blackjack, it seems to end with better results than only exploring 50% of the time.

G. Evaluate your best agent. Evaluate the final trained policy over at least 30 episodes, with exploration disabled. Report mean reward and appropriate additional metrics (e.g., success rate or steps to completion). Compare the results to your baseline. Explain how you selected the best configuration.





Using comparisons across the 3 parameters, I selected the best for each and combined them to train the best agent over 1000 episodes without exploration. The best parameters that I chose were alpha = 0.1, gamma = 0.5, epsilon = 0.5. These were determined to be the best by comparing the mean rewards and win rates. I found that the best agent had a mean reward of -0.093, which is better than the baseline's -0.118. The best agent also has a higher final average reward than the baseline and a better success rate of 43.1%.